シンボリック実行を用いたファームウェアの脆弱性修正検出手法の提案

田中 朝陽[1,a)]　葛野 弘樹[1]　瀧田 愼[1]　白石 善明[1]

概要：IoT 機器の普及に伴い，ファームウェアの脆弱性対策が重要である．しかし，ファームウェアはソースコードが公開されていない場合が多く，更新により脆弱性が修正されたかを第三者が確認することは困難である．バイナリ差分解析により変更箇所を特定可能な場合において，条件分岐の追加や変数値の変更により危険な挙動が生じなくなるかの判断は難しい．本研究では，外部入力に依存する値が特定の条件を満たした場合に危険な挙動が生じる脆弱性を対象として，脆弱性修正前後のファームウェアバイナリから脆弱性修正を検出する手法を提案する．提案手法では，脆弱性修正前後で処理内容が異なる関数を差分関数として抽出し，関数の呼出し関係を用いて，外部入力を取り込む処理から脆弱性に関係する命令や関数呼出しまでの経路のうち，差分関数を経由する経路を解析対象として抽出する．抽出した経路では，外部入力をテイントとして付与し，Reaching Definitions 解析で得られる定義・使用関係を用いて，外部入力から脆弱性に関係する値へのデータ依存関係を追跡する．外部入力への依存が確認された経路にシンボリック実行を適用し，脆弱性に関係する命令や関数呼出しへの到達条件と，CWE ごとに定義した危険状態の成立条件を同時に満たす入力の有無を判定する．脆弱性修正前には条件を満たす入力が存在し，脆弱性修正後には存在しない場合に脆弱性修正を検出する．評価では，OpenWrt に 6 種類の CWE に対応する脆弱性を導入して作成した ARM 向けファームウェアバイナリに提案手法を適用し，全種類で脆弱性修正を検出可能と確認した．

キーワード：ファームウェア，脆弱性修正検出，差分解析，シンボリック実行

Proposal of a Method for Detecting Firmware Vulnerability Fixes Using Symbolic Execution

Tanaka Asahi[1,a)]　Kuzuno Hiroki[1]　Takita Makoto[1]　Shiraishi Yoshiaki[1]

Abstract: With the widespread adoption of IoT devices, addressing vulnerabilities in firmware has become increasingly important. However, firmware source code is often unavailable, making it difficult for third parties to determine whether vulnerabilities have been fixed by firmware updates. Even when binary diffing identifies modified code, it is difficult to determine whether dangerous behavior has been eliminated through added conditional branches or changes to variable values. This study proposes a method for detecting vulnerability fixes from firmware binaries before and after vulnerability remediation, targeting vulnerabilities in which dangerous behavior occurs when a value dependent on external input satisfies a specific condition. The proposed method extracts candidate paths using functions whose processing differs before and after vulnerability remediation and their call relationships, and tracks data dependencies using Reaching Definitions analysis while treating external input as tainted data. Symbolic execution is then applied to candidate paths that depend on external input to determine whether any input simultaneously satisfies the conditions required to reach vulnerability-related instructions or function calls and the dangerous-state conditions defined for each CWE. A vulnerability fix is detected when such an input exists before vulnerability remediation but does not exist after remediation. In the evaluation, the proposed method was applied to ARM firmware binaries created by introducing vulnerabilities corresponding to six CWE categories into OpenWrt, and vulnerability fixes were successfully identified for all six categories.

Keywords: Firmware, Vulnerability Fix Detection, Binary Diffing, Symbolic Execution

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

IoT 機器の普及に伴い，ファームウェアのセキュリティ対策が重要である．IoT 機器やネットワーク機器のファームウェアには，ネットワークやファイルを介して外部入力を受け付けるプログラムが含まれ，入力値の検証不足や不適切なメモリ操作に起因する脆弱性が報告されている [1]．脆弱性の発見後，ベンダは修正版ファームウェアを提供するが，ソースコード，シンボル情報および修正内容が非公開の場合があり，第三者が配布されたバイナリから脆弱性の修正を確認することは困難である [2]．

本研究では，ネットワークを介して攻撃者が制御可能な入力を受け付ける IoT 機器を対象とし，外部入力の影響により成立する脆弱性に着目する．外部入力に起因する脆弱性では，攻撃者が制御可能な入力がプログラム内部へ伝搬し，メモリ操作，算術演算およびコマンド実行に使用されることで脆弱性の成立条件を満たす可能性がある．本研究では，外部入力の影響を受けた場合に脆弱性の成立につながり得る関数呼出しや命令を危険候補シンクとする．

バイナリ差分解析では，脆弱性修正前後で命令列や制御フローが変更された関数を特定可能である．一方，ファームウェア更新には機能追加や不具合修正も含まれるため，変更された関数のみから脆弱性修正に関係する変更を判定することは難しい．また，脆弱性修正は入力検証や境界検査の追加として実装される場合があり [3]，危険候補シンクとなる関数呼出しや命令が脆弱性修正後も残る場合がある．変更箇所や危険候補シンクの有無のみでは脆弱性修正を判断できず，脆弱性の成立可能性を脆弱性修正前後で特定する必要がある．以上より，脆弱性修正の検出には次の課題がある．

課題：**脆弱性修正前後における脆弱性の成立可能性の特定**

> 従来のバイナリ差分解析では，脆弱性修正前後で変更された関数を特定可能な一方，外部入力が危険候補シンクで使用される値に影響し，脆弱性の成立条件を満たす実行経路が存在するかは判定できない．特に，入力検証や境界検査が追加された場合，危険候補シンクが脆弱性修正後も残るため，危険候補シンクの有無のみでは脆弱性修正を判定できない．脆弱性修正を検出するには，外部入力から危険候補シンクまでのデータ依存関係と実行経路を解析し，脆弱性の成立条件を満たす実行状態の有無を脆弱性修正前後で特定する必要がある．

課題を解決するため，本研究では，攻撃者が制御可能な外部入力を起点として成立する脆弱性を対象とする．外部入力に起因する脆弱性では，外部入力が危険候補シンクで使用される値に影響し，CWE ごとの脆弱性の成立条件を満たすことで脆弱性が成立する．外部入力から危険候補シンクまでのデータ依存関係と実行経路を解析することで，脆弱性の成立可能性を脆弱性修正前後で比較可能である．

本研究では，脆弱性修正前のファームウェアを脆弱版，脆弱性修正後のファームウェアを修正版とする．脆弱版と修正版のファームウェアバイナリに対して脆弱性の成立可能性を解析し，解析結果の変化から脆弱性修正を検出する手法を提案する．

提案手法では，バイナリ差分解析から変更された関数を抽出し，外部入力ソースから危険候補シンクまでの関数呼出し経路のうち，変更された関数を経由する候補経路を抽出する．候補経路に Reaching Definitions（RD）解析を適用し，危険候補シンクで使用される値が外部入力に依存するか判定する．

外部入力依存が確認された候補経路にシンボリック実行を適用して危険候補シンクへの到達条件を取得し，CWE ごとの脆弱性成立条件と組み合わせて SMT ソルバで充足可能性を判定する．脆弱性修正前後の充足可能性を比較し，脆弱版では脆弱性成立条件を満たす入力が存在し，修正版では脆弱性成立条件を満たす入力が存在しない場合に，脆弱性が修正されたと判定する．

評価では，OpenWrt 24.10.5 を基に作成した ARM 向けファームウェアの `uhttpd` を対象として，CWE-78，CWE-119，CWE-123，CWE-190，CWE-369 および CWE-416 の 6 種類の脆弱性修正に提案手法を適用した．6 種類すべてにおいて，脆弱版では危険状態の成立条件が SAT となり，修正版では UNSAT となることを確認した．評価結果から，危険状態の成立可能性の変化を用いて脆弱性修正を検出可能なことを確認した．

本研究の主な貢献を以下に示す．

(1) 脆弱性修正前後で変更された関数を基準として解析対象を絞り込み，RD 解析による外部入力依存の判定と，シンボリック実行および SMT ソルバによる危険状態の充足可能性判定を組み合わせることで，脆弱性修正前後における危険状態の成立可能性をバイナリから特定し，成立可能性の変化から脆弱性修正を検出する手法を設計，実装した．

(2) OpenWrt 24.10.5 を基に作成した ARM 向けファームウェアを用いて 6 種類の CWE に対応する脆弱性修正を評価し，6 種類すべてで脆弱版では危険状態が成立可能であり，修正版では成立しないことを特定した．評価結果から，提案手法により脆弱性修正を検出可能なことを確認した．

[1] 神戸大学大学院 工学研究科
Graduate School of Engineering, Kobe University
[a)] asahi.tanaka@gsuite.kobe-u.ac.jp

© 2026 Information Processing Society of Japan

表 1　対象とする脆弱性と脆弱性が成立する状態

CWE-ID	脆弱性タイプ	脆弱性が成立する状態
CWE-78	OS コマンドインジェクション	外部入力を含むコマンド文字列により，追加のコマンドを実行可能な状態
CWE-119	バッファオーバーフロー	バッファ境界の外側へ書込み可能な状態
CWE-123	任意のメモリ領域への書込み	書込み先アドレスと書込み値の双方が外部入力により制御可能な状態
CWE-190	整数オーバーフロー	整数演算の結果がデータ型で表現可能な範囲を超える状態
CWE-369	ゼロ除算	除算の除数が 0 となる状態
CWE-416	解放済みメモリの使用	解放済み領域を指すポインタが解放後に使用される状態

2. 背景

2.1 ファームウェア

ファームウェアは，IoT 機器や組込み機器のハードウェアを制御し，通信や機器制御を担うソフトウェアである．ネットワーク機器では，Linux カーネル，ライブラリ，設定ファイルおよびユーザ空間プログラムを含むファイルシステムがファームウェアとして提供されることがある．ユーザ空間プログラムには，Web サーバやネットワークサービスのように外部入力を受け付けるものが含まれる．

IoT 機器では，機器ベンダがファームウェアを更新することで，機能追加，不具合修正および脆弱性修正を行う．ファームウェアは実行用のバイナリとして配布され，ソースコードやデバッグ用シンボル情報が提供されない場合がある [1]．第三者が更新内容を確認する場合，配布されたファームウェアに含まれるバイナリを解析する必要がある．また，一つのファームウェアには複数の実行ファイルが含まれるため，脆弱性に関係するユーザ空間プログラムを特定し，対象の実行ファイルを解析する必要がある．

2.2 外部入力に起因する脆弱性

IoT 機器上のプログラムは，ネットワーク要求やファイル，標準入力，環境変数およびコマンドライン引数から値を取得して処理する．外部から与えられ，攻撃者が内容を制御可能な入力を外部入力と呼ぶ．

外部入力は，プログラム内部で関数引数や戻り値として受け渡され，演算，条件分岐，メモリアクセスおよび関数呼出しに利用される．入力検証や境界検査が不十分な場合，OS コマンドインジェクション，バッファオーバーフローおよびゼロ除算などの脆弱性が成立する可能性がある．

CWE（Common Weakness Enumeration）は，ソフトウェアに含まれる弱点を種類ごとに分類する体系であり，CWE-ID で弱点の種類を識別する [4]．同じ CWE に分類される脆弱性では，関数や命令が異なる場合でも，脆弱性が成立する状態に共通する特徴がある．本研究では，攻撃者が制御可能な外部入力に起因する 6 種類の脆弱性を対象とする．対象とする脆弱性と脆弱性が成立する状態を表 1 に示す．OS コマンドの実行，メモリ操作，算術演算およびメモリ解放後の使用に関係する脆弱性が含まれ，脆弱性の種類ごとに脆弱性が成立する状態が異なる．

3. 想定モデル

3.1 解析対象

解析対象は，脆弱性修正前後のファームウェアに含まれるユーザ空間実行ファイルである．本研究では，脆弱性修正前のファームウェアを脆弱版，脆弱性修正後のファームウェアを修正版とする．脆弱版と修正版では，対応するユーザ空間実行ファイルを解析し，両者が同一の CPU アーキテクチャで動作することを前提とする．

3.2 脅威モデル

本研究では，ネットワークに接続された IoT 機器が脆弱性を含むファームウェアで動作し，ファームウェア内のユーザ空間プログラムが外部入力を受け付ける状況を想定する．攻撃者は，機器が提供するネットワークサービスや入力インタフェースを通じて，内容を制御可能な外部入力を対象プログラムへ与える．攻撃者はファームウェアの改変，解析対象プログラムの置換および機器内部への物理的なアクセスを行わず，対象プログラムが通常提供する入力経路を利用するものとする．

ユーザ空間プログラムは外部入力を取得し，取得した値を解析，演算，メモリ操作および関数呼出しに利用する．入力検証や境界検査が不十分な場合，外部入力の内容に応じて脆弱性が成立し，OS コマンドの実行，メモリ破壊およびサービス停止などの被害が発生する可能性がある．

図 1 に想定する攻撃シナリオを示す．攻撃者は，ネットワークサービスや入力インタフェースを通じて細工した外部入力を IoT 機器へ与える．ファームウェア内のユーザ空間プログラムは外部入力を取得し，入力処理を行う．脆弱版では，入力検証や境界検査が不十分な場合に脆弱性が成立し，OS コマンドの実行，メモリ破壊，サービス停止などの被害に至る．修正版では，入力検証や境界検査の追加，脆弱性が成立する状態を生じさせる処理の変更により，脆弱性が成立しなくなる状況を想定する．

4. 提案手法

4.1 提案手法の概要

提案手法は，脆弱版と修正版の実行ファイルを解析し，外部入力に起因する危険状態の成立可能性を脆弱性修正前

© 2026 Information Processing Society of Japan

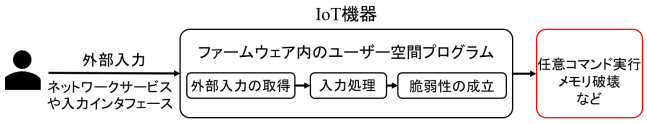


図 1 想定する攻撃シナリオ

後で比較することで，脆弱性修正を検出する．

提案手法の概要を図 2 に示す．提案手法への入力は，脆弱版と修正版の実行ファイル，対象 CWE，外部入力ソース，危険候補シンクおよび CWE ごとの危険状態の定義である．ファームウェアの取得と展開，解析対象となる実行ファイルの抽出は完了していることを前提とする．

最初に，各実行ファイルから関数や関数呼出し関係を取得し，脆弱版と修正版の比較から変更された関数を抽出する．次に，変更された関数を基準として，外部入力ソースから危険候補シンクまでの候補経路を抽出する．候補経路について RD 解析とテイント伝搬を行い，危険候補シンクで使用される値が外部入力に依存するか判定する．外部入力依存が確認された候補経路にシンボリック実行を適用し，危険候補シンクへの到達条件と CWE ごとの危険状態を表す制約を生成する．最後に，両条件の充足可能性を判定し，脆弱版と修正版の結果を比較して脆弱性修正を判定する．提案手法は次の 4 段階で構成する．

(1) バイナリ解析情報と差分関数集合の抽出
(2) 差分関数を基準とした候補経路の抽出
(3) 外部入力依存の判定
(4) 制約充足可能性判定と脆弱性修正前後の比較

関数対応と差分抽出には BinDiff，RD 解析とシンボリック実行には angr，制約充足可能性判定には Z3 を利用する．

4.2 バイナリ解析情報と差分関数集合の抽出

脆弱性修正は，入力検証や境界検査の追加，危険な処理の変更として実行ファイル内の一部の関数に現れる．一方，ファームウェア更新では脆弱性修正以外の変更も含まれるため，実行ファイル全体を後続解析の対象とすると解析候補が増加する．提案手法では，脆弱性修正前後で変更された関数を抽出し，変更された関数を基準として後続の解析範囲を限定する．

脆弱版の実行ファイルを b_{v}，修正版を b_{f} とする．各実行ファイルから，関数と基本ブロック，関数呼出し関係，呼出し箇所，呼出し先および命令とオペランドを取得し，関数を頂点，関数呼出しを有向辺とするコールグラフ $G = (V, E)$ を構築する．構築したコールグラフは，後続の候補経路探索に利用する．

外部入力ソースについては，入力された定義とバイナリ解析結果を照合し，外部入力を取得する API や外部入力が与えられる関数引数を特定する．特定した外部入力ソースごとに，外部入力ソースを含む関数，位置，外部入力が格納される引数，戻り値およびメモリ領域を記録する．危険候補シンクについても対象 CWE の定義と照合し，危険候補シンクを含む関数，位置および危険状態の判定に用いる引数またはオペランドを記録する．

脆弱性修正前後の関数対応には BinDiff を用いる．対応する関数の命令列または制御フローに差異がある場合と，一方にのみ関数が存在する場合を差分関数とし，脆弱版と修正版の差分関数集合をそれぞれ F_{Δ}^{v}，F_{Δ}^{f} とする．

4.3 差分関数を基準とした候補経路の抽出

外部入力ソースと危険候補シンクの全組合せを解析すると，脆弱性修正と関係しない経路も解析対象となる．提案手法では，差分関数を基準として探索範囲を限定し，後続の解析対象となる候補経路を抽出する．

最初に，対象 CWE の外部入力ソースと危険候補シンクをバイナリ解析結果と照合し，各実行ファイル中の位置を特定する．外部入力ソースは，含まれる関数，呼出し位置，外部入力が格納される引数，戻り値およびメモリ領域を記録する．危険候補シンクは，含まれる関数，位置および危険状態の判定に用いる引数またはオペランドを記録する．

次に，差分関数 d を基準として，コールグラフを逆方向へ最大 d_{up}，順方向へ最大 d_{down} 探索する．実装では $d_{\mathrm{up}} = d_{\mathrm{down}} = 4$ とする．探索範囲内で，外部入力ソースを含む関数から危険候補シンクを含む関数までの関数呼出し経路を候補経路として抽出する．同一関数の重複を許さず，外部入力ソース，危険候補シンクおよび関数列が同一の候補を除外する．

候補経路は関数呼出し関係に基づくため，外部入力の値が危険候補シンクまで伝搬することは保証しない．値の伝搬は後続の外部入力依存解析で判定する．

4.4 外部入力依存の判定

危険候補シンクが候補経路上に存在しても，危険状態の成立に関係する値が外部入力の影響を受けなければ，攻撃者が制御する入力によって脆弱性は成立しない．候補経路上の外部入力ソースから危険候補シンクまでのデータ依存関係を解析し，外部入力依存が確認された候補のみをシンボリック実行の対象とする．

4.4.1 RD 解析とテイント伝搬

候補経路に含まれる関数へ angr の RD 解析を適用し，値の定義と使用の関係を取得する．外部入力ソースに由来する値をテイントとして，取得した依存関係に基づき危険候補シンクまで伝搬させる．

テイントの起点は外部入力の取得方法に応じて設定する．入力 API がバッファへデータを書き込む場合は対象メモリ領域，戻り値として返す場合は戻り値，関数引数から取得する場合は対応する仮引数を起点とする．危険候補シンクでは，関数呼出しの場合は対象引数，命令の場合は対象オペランドを観測する．複数の処理による状態変化を対象

© 2026 Information Processing Society of Japan

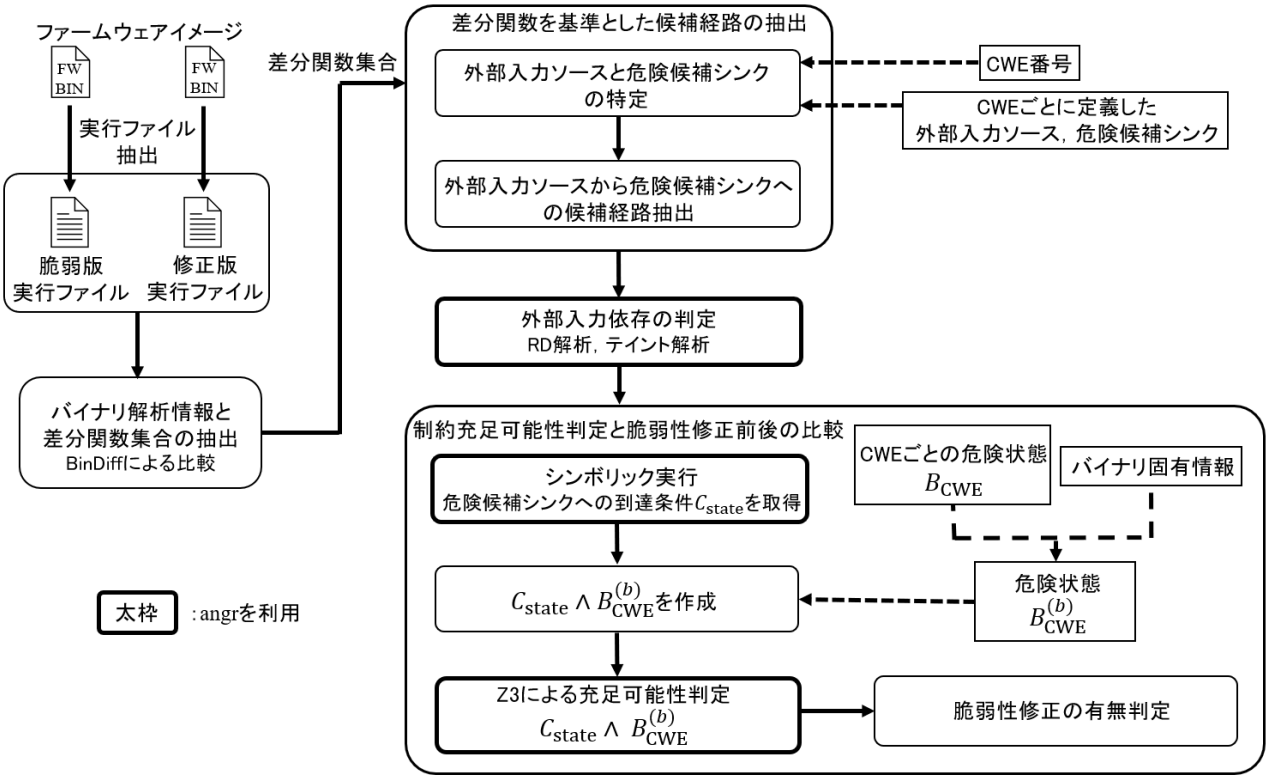


図 2 提案手法の概要

とする場合は，ポインタとメモリ領域の状態も解析する．

関数間では，候補経路上の呼出し箇所について実引数と仮引数を対応付けてテイントを伝搬する．ARM 32 ビットでは，第 1 引数から第 4 引数を `r0`–`r3` に対応付ける．メモリを介する依存はアドレスとバイト範囲で管理し，コピー時には依存関係をコピー先へ伝搬し，上書き時には対象範囲の依存関係を除外する．

4.4.2 外部入力依存の判定基準

外部入力依存の判定結果は，依存あり，依存なしおよび判定不能の 3 種類とする．テイントが危険候補シンクの観測値へ到達した場合を依存ありとする．解析に必要な情報を取得した上でテイントが到達しない場合を依存なしとする．間接呼出し先，メモリアドレスおよび観測値を解決できない場合は判定不能とし，脆弱性修正を確定する根拠には用いない．依存ありと判定した候補経路のみを後続のシンボリック実行へ入力する．

4.5 制約充足可能性判定と脆弱性修正前後の比較

外部入力依存が確認されても，危険候補シンクへ到達する実行経路が存在し，かつ対象 CWE の危険状態を成立させる入力が存在するとは限らない．シンボリック実行により危険候補シンクへの到達条件を取得し，危険状態制約と組み合わせ，両条件を満たす外部入力の存在を判定する．

危険候補シンク q へ到達した実行状態 s が保持する制約集合から，危険候補シンクへの到達条件 $C_{\text{state}}(s)$ を生成する．危険状態制約 $B^{(b)}_{\text{CWE}}(q,s)$ と論理積で組み合わせ，次式を生成する．

$$\Phi^{(b)}(q,s) = C_{\text{state}}(s) \wedge B^{(b)}_{\text{CWE}}(q,s)$$

$\Phi^{(b)}(q,s)$ を Z3 へ入力し，判定結果を以下の 3 種類に分類する．

- **条件が充足可能（SAT）**：$\Phi^{(b)}(q,s)$ を満たす外部入力が存在する．危険候補シンクへの到達条件と対象 CWE の危険状態を同時に満たすことが可能であると判定する．Z3 のモデルから条件を満たす入力例を取得する．
- **条件が充足不可能（UNSAT）**：$\Phi^{(b)}(q,s)$ を満たす外部入力が存在しない．危険候補シンクへの到達条件と対象 CWE の危険状態を同時に満たすことはできないと判定する．
- **判定不能（UNKNOWN）**：探索時間，探索ステップ数，状態数，SMT ソルバの処理時間の上限到達，または解析に必要な情報の不足により，SAT または UNSAT を確定できない状態とする．

複数の実行状態が危険候補シンクへ到達した場合は，一つ以上の実行状態で SAT となれば危険状態は成立可能とする．危険候補シンクへ到達したすべての実行状態が UNSAT となり，設定した探索範囲の探索を完了した場合は，危険状態は成立しないと判定する．

© 2026 Information Processing Society of Japan

5. 評価

5.1 評価項目

提案手法の有効性を確認するため，以下に示す 3 項目について評価を行う．

- **脆弱性修正の検出可否：**脆弱版と修正版を同一条件で解析し，脆弱版で危険状態が成立可能，修正版で成立不可能となることを確認する．危険状態の成立可能性の変化に基づき，脆弱性修正の検出可否を評価する．
- **解析時間および探索規模：**シンボリック実行の解析時間，探索ステップ数，最大同時状態数を測定し，危険状態の成立可能性を判定するために必要な解析時間と探索規模を評価する．
- **解析に必要な情報の取得方法：**提案手法の実行に必要な情報について，自動取得可能な情報と解析者による指定を要する情報を整理し，現実装における自動化範囲を確認する．評価結果の詳細は付録 A.2 に示す．

5.2 評価対象

評価対象には，OpenWrt 24.10.5 を基に作成した ARM 向けファームウェアを用いる．Web サーバの `uhttpd` を解析対象とし，CWE-78，CWE-119，CWE-123，CWE-190，CWE-369 および CWE-416 の 6 種類について脆弱版と修正版を作成した．脆弱性を含む処理は，Juliet Test Suite[5] の実装を参考にした．

修正版では危険候補シンクに相当する処理を可能な範囲で残し，主に以下の変更により危険状態の成立を防止する．

- CWE-78：危険な入力を除外する入力検証を追加
- CWE-119：コピー長に対する境界検査を追加
- CWE-123：書込み可能な範囲を制限
- CWE-190：整数オーバーフローを防止する条件分岐を追加
- CWE-369：除数が 0 の場合に除算を実行しない条件分岐を追加
- CWE-416：メモリアクセス後にメモリを解放するよう処理順序を変更

脆弱版と修正版は対象 CWE に関係する変更を除いて同一条件でビルドし，実行条件や値の制約の変化から脆弱性修正を検出可能か評価する．

5.3 評価環境

評価は KVM の仮想マシン上で実施した．評価環境を表 2 に示す．

候補経路抽出の最大探索深さは $d_{\mathrm{up}} = 4$，$d_{\mathrm{down}} = 4$ とした．シンボリック実行は最大 3000 ステップ，最大同時状態数 256，タイムアウト 60 s とし，Z3 のタイムアウトは 5 s とした．CWE-78 では，コマンド文字列を最大 32 バイト，記号化する外部入力を最大 8 バイト，危険候補シンク到達状態の保存上限を 32 とした．各 CWE の危険状態判定に必要な情報は，評価対象に応じて解析条件として設定した．

表 2 評価環境

項目	内容
ホスト CPU	Intel Core i7-11700 @ 2.50 GHz
解析 VM	4 vCPU，8 GiB
OS	Debian GNU/Linux 13.1
Python	3.13.5
angr	9.2.175
Z3	4.13.0

表 3 脆弱性修正検出結果

CWE-ID	脆弱性タイプ	脆弱版	修正版
CWE-78	OS コマンドインジェクション	SAT	UNSAT
CWE-119	バッファオーバーフロー	SAT	UNSAT
CWE-123	任意のメモリ領域への書込み	SAT	UNSAT
CWE-190	整数オーバーフロー	SAT	UNSAT
CWE-369	ゼロ除算	SAT	UNSAT
CWE-416	解放済みメモリの使用	SAT	UNSAT

5.4 評価結果

5.4.1 脆弱性修正の検出可否

6 種類の CWE について，脆弱版と修正版における危険状態の充足可能性を判定した結果を表 3 に示す．各条件を 5 回解析し，すべての試行で同一の判定結果を得た．

表 3 より，6 種類すべてで脆弱版は SAT，修正版は UNSAT となることを確認した．また，脆弱版では危険候補シンクへの到達条件と危険状態を同時に満たす外部入力が存在する一方，修正版では両条件を同時に満たす外部入力が存在しないことを確認した．以上より，6 種類すべてで危険状態の成立可能性の変化から脆弱性修正を検出でき，すべての実行で設定した解析条件内の探索を完了した．

5.4.2 解析時間および探索規模

シンボリック実行の解析時間と探索規模を表 4 に示す．解析時間は 5 回の平均値と標準偏差，探索ステップ数と最大同時状態数は 5 回に共通する値である．

表 4 より，平均解析時間は 1.55 s から 6.13 s であり，最大同時状態数は 1 状態から 12 状態であることを確認した．また，平均解析時間と最大同時状態数の最大値はいずれも CWE-78 の修正版で確認した．CWE-369 の修正版では解析時間が 4.09 ± 2.28 s となり，他の評価対象より標準偏差が大きい．一方，探索ステップ数と最大同時状態数は 5 回の試行で同一であり，探索した経路の規模に試行間の変化がないことを確認した．

© 2026 Information Processing Society of Japan

表 4　シンボリック実行の解析時間と探索規模

CWE-ID	版	時間 [s]	ステップ数	最大同時状態数
CWE-78	脆弱版	4.75 ± 0.57	62	9
	修正版	6.13 ± 0.25	66	12
CWE-119	脆弱版	1.55 ± 0.10	2	1
	修正版	1.64 ± 0.11	3	2
CWE-123	脆弱版	1.94 ± 0.14	82	1
	修正版	1.99 ± 0.10	83	2
CWE-190	脆弱版	1.84 ± 0.04	73	1
	修正版	1.85 ± 0.09	74	2
CWE-369	脆弱版	2.39 ± 0.12	117	5
	修正版	4.09 ± 2.28	117	7
CWE-416	脆弱版	3.77 ± 0.38	75	3
	修正版	3.78 ± 0.34	74	3

6. 考察

6.1 脆弱性修正検出結果に対する考察

評価した 6 種類すべての CWE で，5 回の解析すべてにおいて脆弱版は SAT，修正版は UNSAT となり，同一の判定結果を得た．また，全実行で設定した解析条件内の探索を完了した．評価の範囲では，脆弱性修正前後における危険状態の成立可能性の変化を検出可能なことを確認した．

修正版では，CWE-119 の `memcpy`，CWE-369 の除算命令，CWE-416 のメモリアクセスのように，危険候補シンクに相当する処理を可能な範囲で残した．脆弱性修正は危険候補シンクの有無ではなく，入力検証，境界検査，演算条件の制限およびメモリ解放と使用の順序変更により危険状態が成立しないことから検出可能なことを確認した．

また，コマンド文字列，メモリアクセス，整数演算およびメモリの解放と使用を含む 6 種類の CWE に共通の解析手順を適用可能なことを確認した．CWE ごとに危険状態を定義することで，異なる種類の脆弱性修正を共通の枠組みで扱える可能性を示した．一方，評価対象は OpenWrt の uhttpd へ評価用に導入した脆弱性であり，実製品の脆弱性修正に対する有効性には追加評価が必要である．

6.2 解析時間と探索規模に対する考察

平均解析時間は 1.55 s から 6.13 s，最大同時状態数は 1 状態から 12 状態であることを確認した．探索ステップ数と最大同時状態数は各条件の 5 回の解析で同一であり，全 60 回の解析は探索上限に到達せず完了した．

提案手法では，差分関数を基準に候補経路を抽出し，外部入力依存を確認した候補のみをシンボリック実行の対象とする．解析対象を段階的に限定することで探索規模を抑制可能であると考えられる．一方，解析範囲を制限しない場合との比較は実施していないため，解析対象の限定による効果の定量評価は今後の課題である．

探索ステップ数と解析時間は必ずしも比例しない．CWE-369 では脆弱版と修正版がともに 117 ステップである一方，平均解析時間は 2.39 s と 4.09 s であることを確認した．解析時間には，探索ステップ数に加え，同時状態数や生成される制約の複雑さが影響すると考えられる．今後は，分岐数が多いプログラムや，記号化した外部入力に依存する演算やメモリアクセスが多いプログラムを対象として，探索規模と解析時間への影響を評価する必要がある．

6.3 提案手法の適用範囲と限界

提案手法は，外部入力に依存する値が危険候補シンクへ伝搬し，危険状態の成立条件を論理式で表現可能な脆弱性を対象とする．評価では，CWE-78，CWE-119，CWE-123，CWE-190，CWE-369 および CWE-416 について充足可能性を判定可能なことを確認した．

現時点の主な制約を以下に示す．

- **解析者による情報指定：**解析開始位置，メモリ領域，外部入力の与え方，危険状態制約の生成に必要な情報の一部を解析者が指定する．脆弱性修正前後の候補経路の対応にも解析者による指定を用いる．評価時に指定した情報を付録 A.2 に示す．
- **CPU アーキテクチャ：**評価は ARM 32 ビットを対象とし，関数間の値の対応には ARM の関数呼出し規約を利用する．他の CPU アーキテクチャでは，各関数呼出し規約への対応が必要である．
- **解析可能なプログラム構造：**複雑なデータ依存やポインタ参照，マルチスレッド処理を含む場合，解析結果を確定できない可能性がある．
- **シンボリック実行の探索上限：**探索ステップ数，同時状態数，解析時間に上限を設定するため，上限内に探索を完了できない場合は判定不能となる．

現実装では，危険状態制約の生成に必要なバイナリ固有情報の取得，外部入力に応じた初期状態の生成，脆弱性修正前後の候補経路の対応付けに解析者による指定を要する点が課題である．

6.4 関連研究との比較

関連研究と本研究の比較を表 5 に示す．SaTC[6]，KARONTE[7]，HermeScan[8] は，単一のファームウェアを対象として外部入力に由来するデータの伝搬を解析し，脆弱性につながる可能性のある処理を検出する．外部入力の影響を解析する点は提案手法と共通するが，脆弱性修正前後における脆弱性の成立可能性の変化は解析しない．

PS^3[9] と REACT[10] は，バイナリの変更を解析し，既知の脆弱性修正が対象バイナリに適用されているか判定する．脆弱性修正に伴うバイナリの変化を利用する点は提案手法と共通するが，外部入力の影響や脆弱性の成立条件を考慮して脆弱性修正前後を比較することを目的としていない．入力検証や境界検査の追加では危険候補シンクが修正

© 2026 Information Processing Society of Japan

表 5　関連研究との比較

研究	解析対象	目的	差分利用	外部入力依存	危険状態の成立判定
SaTC[6]	単一のファームウェア	外部入力に起因する脆弱性候補の検出	–	✓	–
KARONTE[7]	単一のファームウェア	複数バイナリ間のデータ伝搬に起因する脆弱性候補の検出	–	✓	–
HermeScan[8]	単一のファームウェア	外部入力に起因する脆弱性候補の検出	–	✓	–
PS^3[9]	修正情報と対象バイナリ	既知の脆弱性修正が対象バイナリに適用されているかの判定	✓	–	–
REACT[10]	修正前後の情報と対象バイナリ	既知の脆弱性修正が対象バイナリに適用されているかの判定	✓	–	–
提案手法	脆弱版と修正版のファームウェア内実行ファイル	外部入力に起因する脆弱性が修正後に成立しないことの判定	✓	✓	✓

✓：該当あり，–：該当なし

版にも残る場合があるため，コードの変更のみでは脆弱性の成立可能性の変化を判定不可能である．

提案手法では，バイナリ差分を用いて解析対象を限定し，外部入力から危険候補シンクへの依存関係と実行経路を解析する．さらに，到達条件と CWE ごとの危険状態を同時に満たす入力の有無を脆弱性修正前後で比較し，脆弱版で成立する危険状態が修正版では成立しない場合に脆弱性修正と判定する．脆弱性の成立可能性の変化を脆弱性修正前後で解析する点が関連研究と異なる．

7. まとめ

IoT 機器の普及に伴い，ソースコードが公開されていないファームウェアについて，更新後に脆弱性が修正されたかを第三者が確認する手法が求められている．本研究では，脆弱性修正前後のファームウェアから抽出した実行ファイルを対象として，脆弱性修正を検出する手法を提案した．

提案手法は，バイナリ差分解析で抽出した差分関数を中心に外部入力ソースから危険候補シンクまでの候補経路を抽出し，RD 解析により危険候補シンクで使用される値の外部入力依存を判定する．さらに，シンボリック実行と SMT ソルバを用いて CWE ごとの危険状態の成立可能性を判定し，脆弱性修正前後の解析結果を比較する．

OpenWrt の uhttpd を対象として，CWE-78, CWE-119, CWE-123, CWE-190, CWE-369 および CWE-416 の 6 種類を評価した．各条件について 5 回解析した結果，6 種類すべてで脆弱版は SAT，修正版は UNSAT となり，全実行で設定した解析条件下の探索を完了した．評価結果から，対象とした 6 種類の CWE について，危険状態の成立可能性の変化から脆弱性修正を検出可能と確認した．

今後は，解析開始位置やメモリ領域などの手動指定情報の自動取得を進め，実際の製品ファームウェアや異なる CPU アーキテクチャを対象とした評価を行う予定である．

謝辞　本研究の一部は，JSPS 科研費 JP26K21217，JST ACT-X JPMJAX24M4，ならびに経済安全保障重要技術育成プログラム JPMJKP24K2 の支援を受けたものである．

参考文献

[1] Costin, A., Zaddach, J., Francillon, A. and Balzarotti, D.: A Large-Scale Analysis of the Security of Embedded Firmwares, Proc. 23rd USENIX Security Symposium （USENIX Security 14）, pp.95–110, USENIX Association （2014）.

[2] David, Y., Partush, N. and Yahav, E.: FirmUp: Precise Static Detection of Common Vulnerabilities in Firmware, Proc. 23rd International Conference on Architectural Support for Programming Languages and Operating Systems （ASPLOS）, pp.392–404 （2018）.

[3] Zhang, H. and Qian, Z.: Precise and Accurate Patch Presence Test for Binaries, Proc. 27th USENIX Security Symposium （USENIX Security 18）, pp.887–902, USENIX Association （2018）.

[4] The MITRE Corporation: About CWE, Common Weakness Enumeration, The MITRE Corporation （オンライン）, 入手先〈https://cwe.mitre.org/about/index.html〉（参照 2026-08-16）.

[5] NSA Center for Assured Software：Juliet C/C++ 1.3, NIST Software Assurance Reference Dataset（SARD）（オンライン）, 入手先〈https://samate.nist.gov/SARD/test-suites/112〉（参照 2026-08-10）.

[6] Chen, L., Wang, Y., Cai, Q., Zhan, Y., Hu, H., Linghu, J., Hou, Q., Zhang, C., Duan, H. and Xue, Z.: Sharing More and Checking Less: Leveraging Common Input Keywords to Detect Bugs in Embedded Systems, Proc. 30th USENIX Security Symposium （USENIX Security 21）, pp.303–319, USENIX Association （2021）.

[7] Redini, N., Machiry, A., Wang, R., Spensky, C., Continella, A., Shoshitaishvili, Y., Kruegel, C. and Vigna, G.: KARONTE: Detecting Insecure Multi-binary Interactions in Embedded Firmware, Proc. 2020 IEEE Symposium on Security and Privacy （S&P）（2020）.

[8] Gao, Z., Zhang, C., Liu, H., Sun, W., Tang, Z., Jiang, L., Chen, J. and Xie, Y.: Faster and Better: Detecting Vulnerabilities in Linux-based IoT Firmware with Optimized Reaching Definition Analysis, Proc. Network and Distributed System Security Symposium （NDSS 2024）（2024）.

[9] Zhan, Q., Hu, X., Li, Z., Xia, X., Lo, D. and Li, S.: PS^3: Precise Patch Presence Test Based on Semantic Symbolic Signature, Proc. 46th IEEE/ACM International Conference on Software Engineering （ICSE 2024）, pp.1–12（2024）.

[10] Zhan, Q., Hu, X., Xia, X. and Li, S.: REACT: IR-Level Patch Presence Test for Binary, Proc. 39th IEEE/ACM International Conference on Automated Software Engineering （ASE 2024）, pp.381–392 （2024）.

© 2026 Information Processing Society of Japan

表 A.1　評価用ファームウェアに導入した処理

CWE-ID	実行ファイル	変更箇所	脆弱版	修正版
CWE-78	uhttpd	client_parse_header()	HTTP の Host ヘッダを含む文字列をコマンドとして使用可能な処理を導入	危険な文字を除外する入力検証を追加
CWE-119	uhttpd	uh_urldecode()	外部入力に応じた長さで固定長バッファへデータを書き込む処理を導入	書込みサイズがバッファ境界を越えないよう境界検査を追加
CWE-123	uhttpd	client_parse_header()	HTTP ヘッダから取得したアドレスと値を用いてメモリへ書き込む処理を導入	書込み可能なメモリ領域を制限
CWE-190	uhttpd	client_parse_header()	外部入力から取得した整数値を用いてオーバーフロー可能な加算処理を導入	演算前に値の範囲を検査する条件分岐を追加
CWE-369	uhttpd	client_parse_header()	HTTP の Content-Length から取得した値を除数として使用する処理を導入	除数が 0 の場合に除算を行わない条件分岐を追加
CWE-416	uhttpd	client_parse_header()	HTTP ヘッダ処理中に確保したメモリを解放した後に同一領域へアクセスする処理を導入	メモリアクセス後に対象領域を解放するよう処理順序を変更

表 A.2　評価時に解析者が指定した主な情報

CWE-ID	解析者が指定した情報	指定した理由
CWE-78	HTTP ヘッダを外部入力として扱うための情報，シンボリック実行開始時のレジスタとメモリの初期状態	HTTP 要求処理の途中から解析を開始するため，外部入力が格納されるメモリと解析開始時の状態を与える必要があるため
CWE-119	対象バッファの位置と大きさ，関数引数とレジスタの対応	バイナリ解析のみでは，書込み先となるスタック上のオブジェクト境界を一意に特定できないため
CWE-123	書込みを許可するメモリ領域の基準アドレスと大きさ	プログラムの意味に基づく書込み可能領域をバイナリのみから一意に判断できないため
CWE-190	外部入力を与えるためのシンボリック実行の初期状態	対象となる入力値をプログラム内部へ与えた状態から解析を開始するため
CWE-369	解析開始位置，外部入力を与える位置，対象の除算命令と除数に対応するオペランド	除算命令に到達する実行状態と危険状態制約に使用する除数を対応付ける必要があるため
CWE-416	外部入力を与えるための情報，解析時に使用する入力長の条件	外部入力に応じた解放と使用の実行経路をシンボリック実行で再現するため

付　　録

A.1　評価用ファームウェアの作成

評価では，OpenWrt 24.10.5 を基に，CWE-78，CWE-119，CWE-123，CWE-190，CWE-369 ならびに CWE-416 の 6 種類について脆弱版と修正版のファームウェアを作成した．各評価対象では，同一の OpenWrt を基に対象 CWE に関係する処理のみを変更し，脆弱版と修正版を同一条件でビルドした．脆弱性を含む処理の作成には Juliet Test Suite[5] を参考にした．各 CWE で変更した実行ファイルと処理内容を表 A.1 に示す．

脆弱版では，各 CWE の危険状態を外部入力により成立可能な処理を導入した．修正版では，危険候補シンクに相当する処理を可能な範囲で残し，入力検証，境界検査，値の制限，条件分岐ならびに処理順序の変更により危険状態の成立を防止した．危険候補シンクを残すことで，関数や命令の有無ではなく，危険状態の成立可能性の変化から脆弱性修正を判定可能か評価する．

A.2　解析に必要な情報の取得方法

提案手法の実行に必要な情報には，自動取得する情報と著者が指定する情報がある．候補経路の抽出と外部入力依存の判定に用いる情報は自動取得する．一方，シンボリック実行の開始位置，危険状態制約の生成に用いる情報の一部および 6 種類すべての CWE における脆弱版と修正版の候補経路の対応情報は著者が指定した．表 A.2 に，各 CWE で著者が指定した主な情報を示す．

候補経路は各実行ファイルから自動抽出するが，脆弱版と修正版の候補経路の対応付けは現在の提案手法の実装では自動化していない．

著者が指定した情報のうち，命令や関数の位置，オペランド，外部入力の伝搬に関する情報は，バイナリ解析による自動取得を検討可能である．一方，対象バッファの境界や書込み可能なメモリ領域のように，プログラムの意味に依存する情報は，自動取得方法の検討が必要である．

© 2026 Information Processing Society of Japan